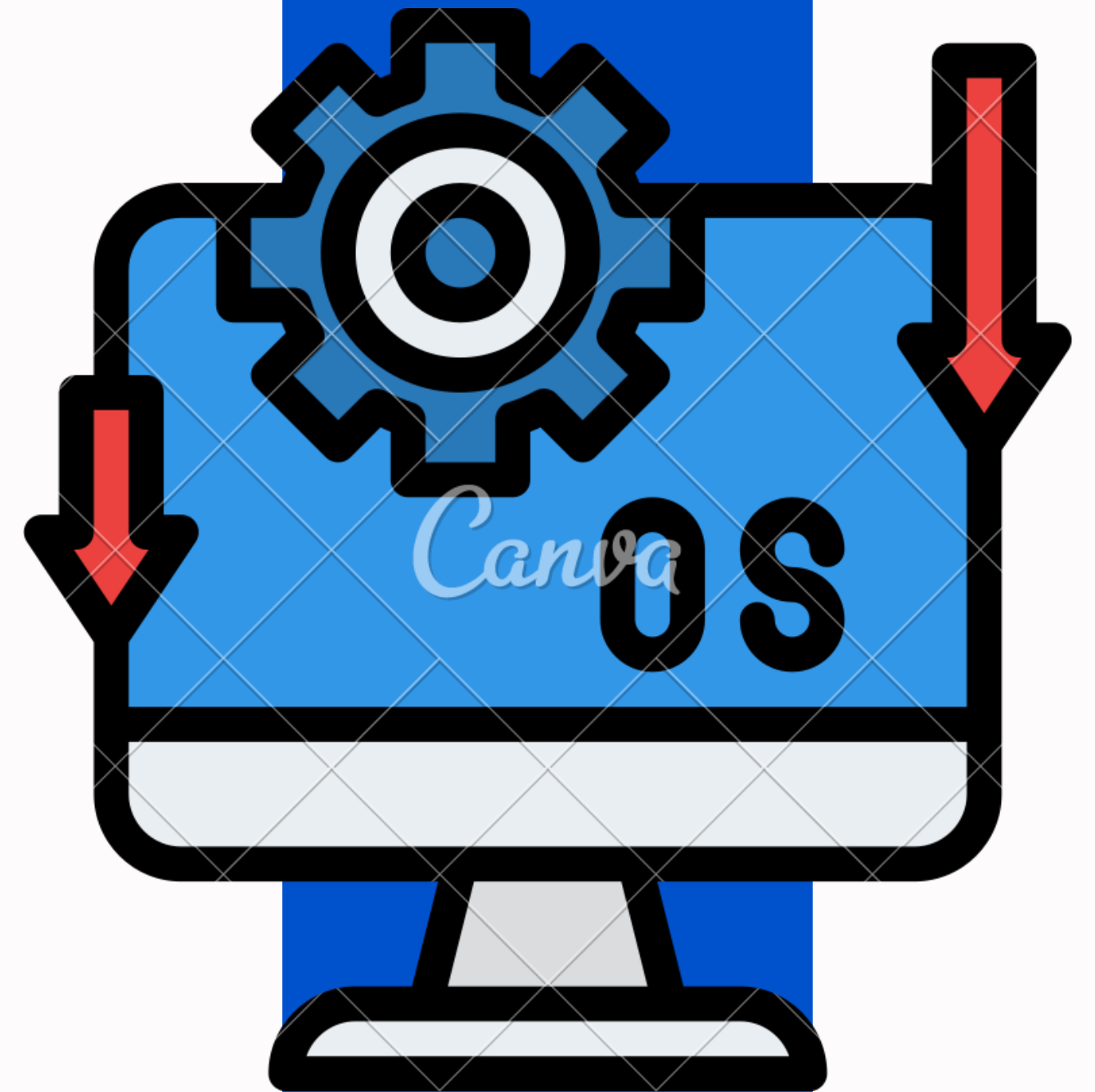


Presentation 2024

OS PRAC SESSION

Komal Tarachandani - 2021600065



- 1. Introduction to processes**
- 2. Basic syntax of processes in C**
- 3. Communication in Processes (IPC and Shared Memory)**
- 4. Implementation of Shared memory via a Chat Application**
- 5. Implementation of IPC**
- 6. How Process Synchronization plays an important part**
- 7. Syntax of Semaphores**
- 8. Process Synchronization in Chat Application using Semaphores**
- 9. Multiple Semaphores example**

What is a Process?

Program in execution

Stack:

The process Stack contains the temporary data such as method/function parameters, return address and local variables.

Heap:

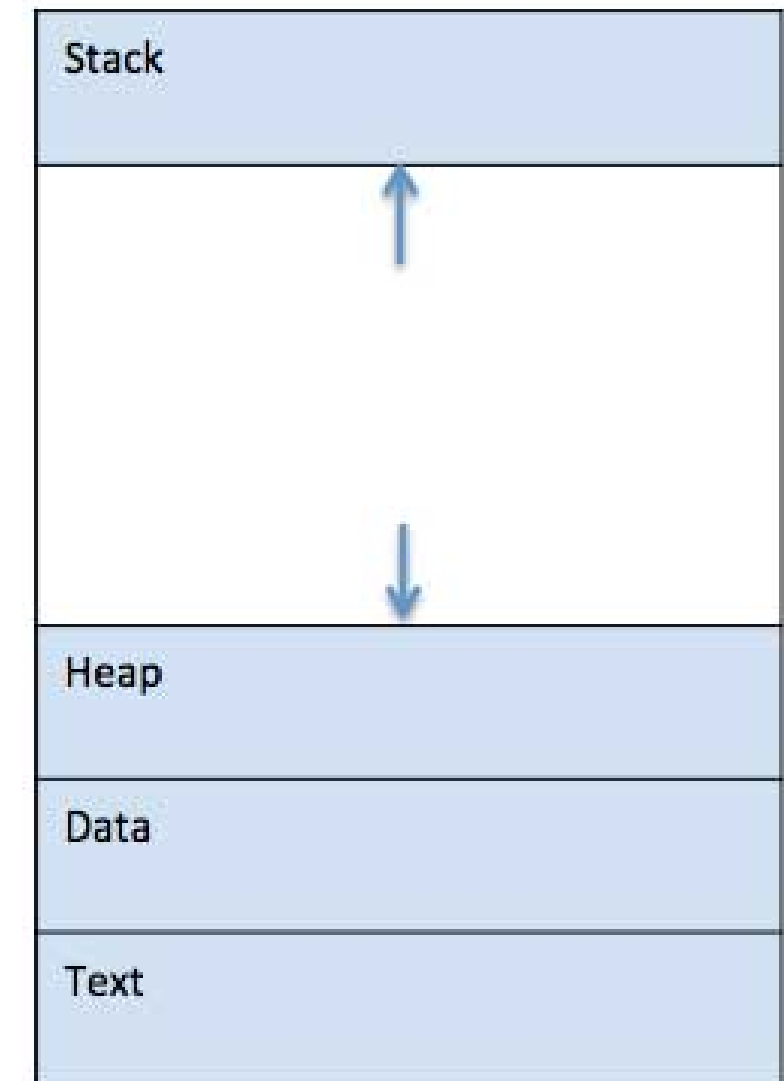
This is dynamically allocated memory to a process during its run time.

Text:

This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.

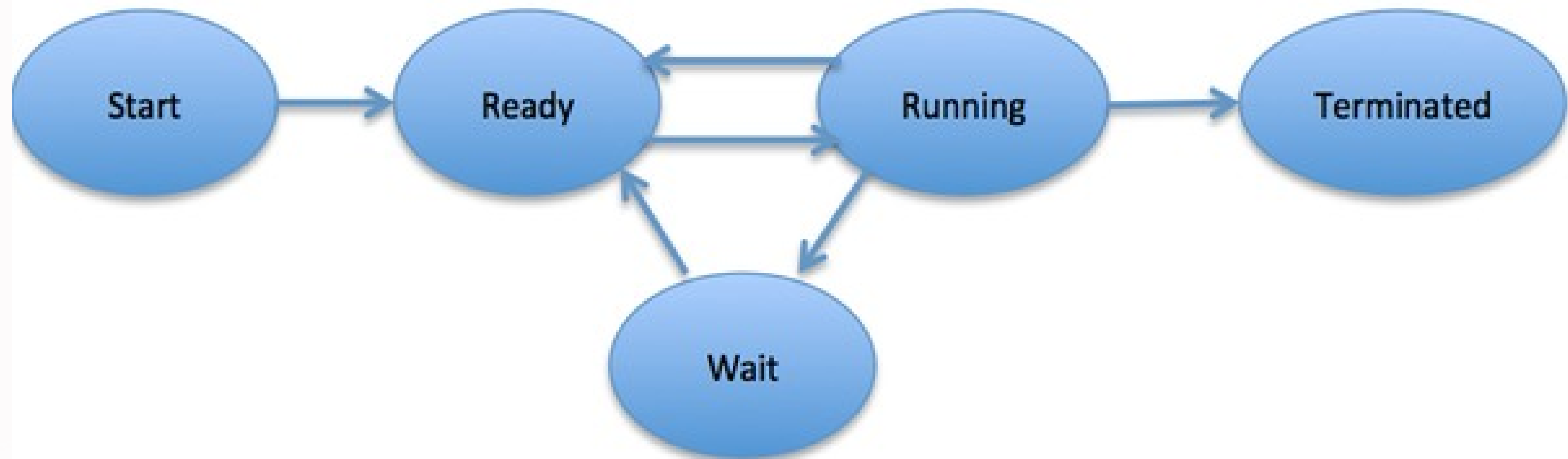
Data:

This section contains the global and static variables.



Process Lifecycle

Process Journey



Basic Syntax of Processes in C



```
int main()
{
    pid_t pid;
    int t=5;
    pid=fork();
    if (pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    } else if (pid == 0) {
        printf("This is the child process\n");
        t+=1;
        printf("child updated t: %d\n",t);
    } else {
        printf("This is the parent process\n");
        t-=1;
        printf("parent updated t: %d\n",t);
    }
    return 0;
}
```

```
This is the parent process
parent updated t: 4
This is the child process
child updated t: 6

...Program finished with exit code 0
Press ENTER to exit console.
```

```
This is the parent process
parent updated t: 4

...Program finished with exit code 0
Press ENTER to exit console.
```


Basic Syntax of Processes in C



```
int main()
{
    pid_t pid;           // custom data type for process ids
    int t=5;             // shared variable
    pid=fork();
    if (pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    } else if (pid == 0) {
        printf("This is the child process\n");
        t+=1;
        printf("child updated t: %d\n",t);
    } else {
        printf("This is the parent process\n");
        t-=1;
        printf("parent updated t: %d\n",t);
    }
    return 0;
}
```

```
This is the parent process
parent updated t: 4
This is the child process
child updated t: 6

...Program finished with exit code 0
Press ENTER to exit console.
```

```
This is the parent process
parent updated t: 4

...Program finished with exit code 0
Press ENTER to exit console.
```


Basic Syntax of Processes in C



```
int main()
{
    pid_t pid;
    int t=5;
    pid=fork();
    if (pid < 0) {
        fprintf(stderr, "Fork failed\n");
        return 1;
    } else if (pid == 0) {
        printf("This is the child process\n");
        t+=1;
        printf("child updated t: %d\n",t);
    } else {
        printf("This is the parent process\n");
        t-=1;
        printf("parent updated t: %d\n",t);
    }
    return 0;
}
```

When a process calls `fork()`, the operating system creates a new process that is an exact copy of the original process. After the `fork()` call, both the parent and child processes continue executing from the point of the `fork()` call, but they have separate memory spaces and run independently.

seperate
memeory

```
33 }
This is the parent process
parent updated t: 4
This is the child process
child updated t: 6

...Program finished with exit code 0
Press ENTER to exit console.
```

```
34
This is the parent process
parent updated t: 4

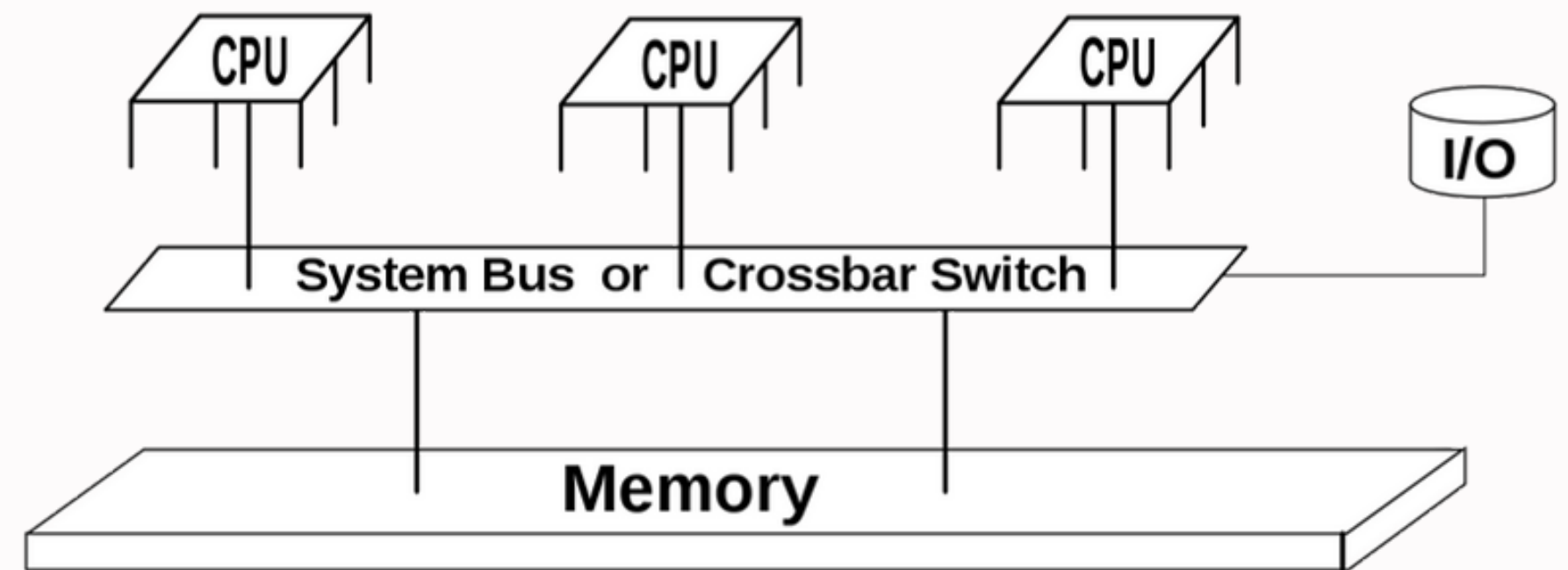
...Program finished with exit code 0
Press ENTER to exit console.
```

Process Communication

Process communication refers to the **exchange of data, signals, or messages** between different processes running **concurrently** on a computer system. Processes typically operate in ***isolated memory spaces***, so communication mechanisms are needed to facilitate interaction and coordination between them



- 1. Shared Memory
- 2. Pipes
- 3. Socket
- etc.



shared memory

Shared Memory in C

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

#define SHM_SIZE 1024

int main()
{
    char *shmaddress;
    int shmid;
    key_t key;
```

standard input output

standard library

specific libraries

access to posix api

preprocessing directive => #define

constant

pointer

unistd data type for key

POSIX

Portable Operating System Interface

basics
↓
UNIX

Shared Memory in C

```
//key banana
key=ftok(".",1);
if (key == -1) {
    perror("ftok");
    exit(1);
}
printf("key successfully created\n");

//shared memory banana
shmid = shmget(key, SHM_SIZE, IPC_CREAT | 0666); //
if (shmid == -1) {
    perror("shmget");
    exit(1);
}
```

→ file to key ("address", id); → '0'

FLAG PERMISSION FOR ACCESS R/W

get shared memory with the defined key or create

Shared Memory in C

```
//attach karlo
shmaddr = shmat(shmids, NULL, 0);
if (shmaddr == (char *) -1) {
    perror("shmat");
    exit(1);
}

//write to shared memory
strcpy(shmaddr, "This is my first time creating shared memory ! yay");

printf("\nData written to shared memory: %s\n", shmaddr);

//delete the shared memory
if (shmctl(shmids, IPC_RMID, NULL) == -1) {
    perror("shmctl");
    exit(1);
}
```

reference address

FLAG

→ att

→ data

Shared Memory with processes

*In the same prog,
copy previous
proccess code*

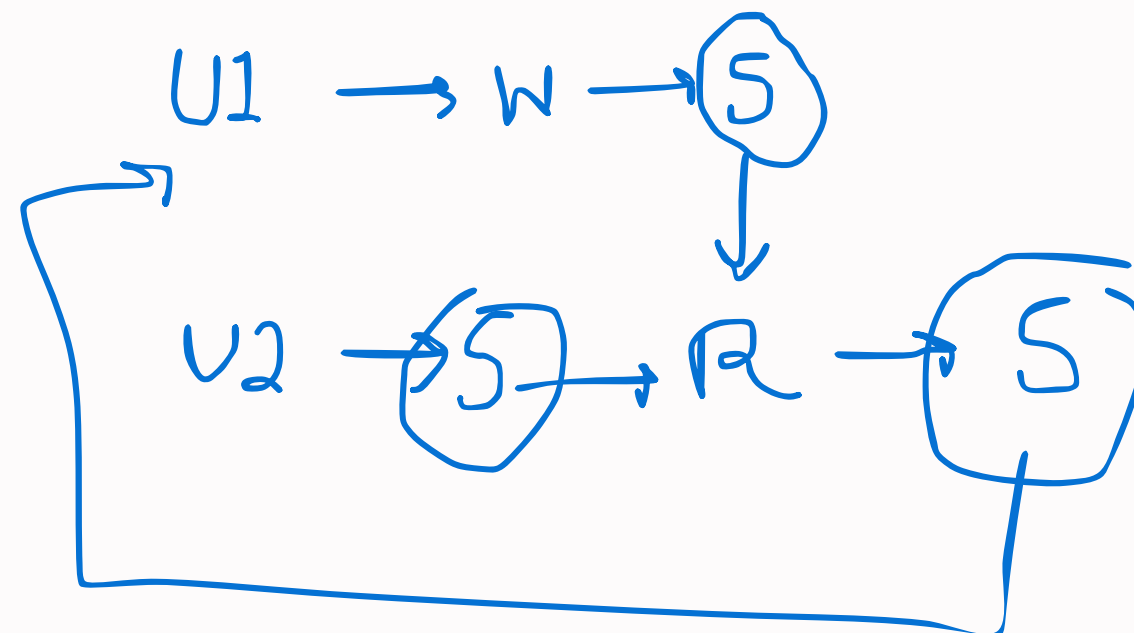
```
pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    exit(1);
} else if (pid == 0) { // Recieve )
    while (1) {
        sleep(5);
        printf("User 2 reads as : %s\n", shmaddr);
        sleep(5);
    }
} else { // Send
    while (1) {
        printf("User 1 writes: ");
        fgets(shmaddr, SHM_SIZE, stdin);
        sleep(5);
    }
}
```


What is sleep()?

It is used to pause the execution of a program for a specified number of seconds

- sleep() takes a single argument, which specifies the number of seconds the program should sleep for.
- After the specified duration has elapsed, the program resumes execution from the point where sleep() was called.



zzZ

Race Condition

```
// Initialize shared variable
*shared_mem = 0;

pid = fork();

if (pid < 0) {
    perror("fork");
    exit(1);
} else if (pid == 0) {

    for (int i = 0; i < 1000000; i++) {
        (*shared_mem)--;
    }
    exit(0);
} else {

    for (int i = 0; i < 1000000; i++) {
        (*shared_mem) += 1;
    }
}
```


Semaphores

- A semaphore is a *synchronization primitive* used in concurrent programming to control access to a shared resource by multiple processes or threads.
- It provides a *mechanism for enforcing limits on the number* of threads or processes that can access the shared resource simultaneously

Atomic operations:

- *Wait(s):*
S.value := S.value - 1;
if S.value < 0
 add this process to S.L;
 block;
end;
- *Signal(s):*
S.value := S.value + 1;
if S.value ≤ 0
 remove a process *P*
 from S.L;
 wakeup(*P*);
end;

Semaphores

```
semid = semget(SEM_KEY, 1, IPC_CREAT | 0666);  
if (semid == -1) {  
    perror("semget");  
    exit(EXIT_FAILURE);  
}  
  
// Initialize semaphore value to 1  
semctl(semid, 0, SETVAL, 1);
```

```
// Semaphore operations  
static void sem_wait(int semid) {  
    struct sembuf sb = {0, -1, 0}; //  
    semop(semid, &sb, 1);  
}  
  
static void sem_signal(int semid) {  
    struct sembuf sb = {0, 1, 0}; //  
    semop(semid, &sb, 1);  
}
```

Semaphores

`sem_wait(int semid)`: This function is intended to be used to wait for a semaphore. It takes a semaphore ID `semid` as its parameter.

`struct sembuf sb = {0, -1, 0};`

1. `sem_num` is set to 0, indicating the index of the semaphore within the semaphore set.
 2. `sem_op` is set to -1, which means it will decrement the value of the semaphore.
 3. `sem_flg` is set to 0, indicating no special flags are set for this operation.
- `semop(semid, &sb, 1);`
This line invokes the `semop` function to perform the semaphore operation. It decrements the value of the semaphore identified by `semid` using the parameters specified in `sb`.
The third parameter 1 indicates that only one semaphore operation is being performed.

```
// Semaphore operations
static void sem_wait(int semid) {
    struct sembuf sb = {0, -1, 0}; //
    semop(semid, &sb, 1);
}

static void sem_signal(int semid) {
    struct sembuf sb = {0, 1, 0}; //
    semop(semid, &sb, 1);
}
```

Semaphores

```
if (pid == 0) {  
    // Child  
    sem_wait(semid); // Wait  
    printf("Process B is executing its critical section.\n");  
    (*shared_memory)+=2;  
    printf("Shared Memory after Process B: %d\n", *shared_memory);  
    sem_signal(semid); // Signal  
} else {  
    // Parent  
    sem_wait(semid); // Wait  
    printf("Process A is executing its critical section.\n");  
    (*shared_memory)-=5;  
    printf("Shared Memory after Process A: %d\n", *shared_memory);  
    sem_signal(semid); // Signal  
    wait(NULL); // Wait for child process to finish  
}
```

Semaphores

```
if (pid == 0) {  
    // Child  
    sem_wait(semid); // Wait  
    printf("Process B is executing its critical section.\n");  
    (*shared_memory)+=2;  
    printf("Shared Memory after Process B: %d\n", *shared_memory);  
    sem_signal(semid); // Signal  
} else {  
    // Parent  
    sem_wait(semid); // Wait  
    printf("Process A is executing its critical section.\n");  
    (*shared_memory)-=5;  
    printf("Shared Memory after Process A: %d\n", *shared_memory);  
    sem_signal(semid); // Signal  
    wait(NULL); // Wait for child process to finish  
}
```


Chat with Semaphores

```
// semaphores
sem_user1 = sem_open("/sem_user1", O_CREAT | O_EXCL, 0644, 0);
if (sem_user1 == SEM_FAILED) {
    perror("sem_open");
    exit(EXIT_FAILURE);
}

sem_user2 = sem_open("/sem_user2", O_CREAT | O_EXCL, 0644, 0);
if (sem_user2 == SEM_FAILED) {
    perror("sem_open");
    exit(EXIT_FAILURE);
}
```

flags to create or open existing initial value

permissions
(octal)

Chat with Semaphores

```
if (pid == 0) {  
    while (1) {  
        sem_wait(sem_user1); // Wait for User 1 to write  
        printf("*****USER 2: USER 1 MESSAGED *****\n");  
        printf("User1: %s", shared_memory);  
        printf("User 2: ");  
        fgets(shared_memory, SHM_SIZE, stdin);  
        sem_post(sem_user2); // Signal User 2 has written  
    }  
} else {  
    while (1) {  
        printf("*****USER 1 : YOUR CHANCE TO TYPE*****\n");  
        if (flag == 1) {  
            printf("User2: %s", shared_memory);  
        }  
        flag = 1;  
        printf("User1: ");  
        fgets(shared_memory, SHM_SIZE, stdin);  
        sem_post(sem_user1); // Signal User 1 has written  
        sem_wait(sem_user2); // Wait for User 2 to read  
    }  
}
```

- 1. Explore storing messages**
- 2. Dining Philosopher problems**
- 3. Threads**
- 4. MultiProcessing and Multithreading**

THANK YOU!

9326227834